경력기술서 원본

> 교육과 프로젝트에서 실제로 맡아 수행한 내용을 정리한 문서입니다. 팀 프로젝트의 전체 기능과 제가 직접 작업한 기능을 구분하고, 완료하지 못한 연동이나 검증은 완료 성과처럼 표현하지 않습니다.

1. 교육 및 전공

SKT K-뉴딜 ALEPH 1기

- 기간: 2026.08 ~ 2026.11 (2026년 11월 기준 과정 진행 중)
- 주요 내용: AI 활용, Python, Linux, API, 자동화, 데이터 분석 및 인프라 보안 실습
- 주요 경험: SENTRA 사내망 다계층 접근통제 캡스톤, T10 타이핑 연구, 개인 앱 및 Studio 과제
- 실습 내용: RBAC, 접근 정책, 권한 요청·승인·회수, 조건부 접근, 이상 징후 탐지

ITWill 백엔드 개발 교육

- 기간: 2024.08 ~ 2025.05
- 주요 내용: Java, JavaScript, Python, Linux 및 웹서비스 개발
- 프로젝트: Stock-Mate, MrBean, Packtory_DE 등 팀 프로젝트
- 경험: Spring MVC / Spring Boot 기반 웹 개발, JSP·Thymeleaf 화면 구현, MyBatis·JPA를 이용한 데이터 처리

인제대학교 나노융합공학부 반도체공학 전공

- 학위: 학사
- 관련 경험: 반도체 공정 실습, 클린룸 실습, 전공 실험
- 프로젝트 경험: Arduino, Bluetooth, MIT App Inventor를 활용한 화재 알림 시제품

---

2. 주요 프로젝트 경험

SENTRA — 사내망 다계층 접근통제 시스템

- **유형:** ALEPH 1기 캡스톤 팀 프로젝트
- **기술:** Python, Flask, PyJWT, SQLite, React, TypeScript
- **저장소:** https://github.com/mingming-01/sentra

프로젝트 개요

SENTRA는 사내 직원의 역할과 소속, 접근하려는 자원에 따라 권한을 판단하는 사내망 다계층 접근통제 시스템이다. 로그인 이후 모든 자료에 접근할 수 있는 구조가 아니라, 서버에서 사용자 정보와 정책을 확인해 요청을 허용하거나 거부하도록 구성했다.

직원 명부와 초대 기반 가입, 계정 상태 관리, 접근 정책, 감사 로그, 기간제 권한 요청·승인·회수 흐름을 함께 다뤘다.

주요 기여 및 수행 내용

백엔드와 데이터 처리 흐름을 중심으로 다음 기능을 작업했다.

- Flask 기반 API 구현 및 프런트엔드와의 요청·응답 연결
- SQLite 데이터베이스 구조 설계 및 사용자·직원 관련 데이터 처리
- 비밀번호 해시를 이용한 로그인 인증과 토큰 발급 흐름 구현
- 계정 상태를 확인하고 비활성화된 계정의 접근을 제한하는 처리
- 직원 명부에 사전 등록된 정보를 사원번호와 일회용 초대 코드로 검증하는 가입 흐름
- 초대 코드 재사용 방지 및 직원 명부와 가입 계정 연결
- 사용자 마이페이지 관련 데이터베이스 및 API 처리
- 직원의 기간제 권한 요청, 운영자의 승인·반려, 승인된 권한의 만료·회수 흐름 구현 및 연결

문제 해결 및 검증

직원 가입과 계정 상태 관리

직원이 가입할 때 이름이나 소속을 임의로 입력하는 대신, 관리자에 의해 등록된 직원 명부와 사원번호·일회용 초대 코드를 기준으로 계정을 연결하도록 구성했다. 초대 코드를 한 번 사용한 뒤 다시 사용할 수 없는지, 직원 계정을 비활성화하면 로그인과 보호 API 접근이 제한되는지 확인했다.

기간제 권한의 생명주기

권한 요청을 제출했다고 바로 접근이 허용되면 안 되므로, 승인 전에는 기존 정책을 유지하도록 했다. 운영자가 승인하면 정해진 기간 동안 예외 권한이 적용되고, 기간이 끝나거나 관리자가 회수하면 다시 기존 정책으로 돌아가는 흐름을 연결했다. 요청 버튼이 동작하는 것뿐 아니라 승인 전후와 회수 후의 접근 판정이 달라지는지 확인했다.

서버 중심의 접근통제

화면에서 메뉴를 숨기는 것만으로 접근통제가 완성되지는 않는다. 사용자가 API를 직접 호출하더라도 서버가 인증과 정책을 확인하도록 구성했고, 접근이 거부될 때는 판정 사유와 권한 요청 흐름을 화면에서 안내하도록 연결했다.

결과 및 배운 점

직원 가입, 초대 코드 재사용 방지, 비활성 계정 차단, 권한 요청·승인·회수 등 주요 시나리오를 실행하며 인증과 인가, 계정 상태, 정책 판정이 서로 연결되는 흐름을 경험했다. 특히 권한은 부여하는 시점뿐 아니라 만료와 회수까지 관리해야 한다는 점을 구현과 테스트를 통해 배웠다.

Packet Tracer 장비와 백엔드의 직접 연동, Syslog/NetFlow 수집, Discord 알림의 실제 수신은 애플리케이션에서 구현한 기능과 별도의 검증 범위로 구분한다.

Maple Growth Tracker — 게임 성장 데이터 웹서비스

- **유형:** 개인 프로젝트
- **기술:** Python, Flask, HTML, CSS, JavaScript, Chart.js, NEXON Open API, Gemini API, OpenRouter, SQLite, PostgreSQL(Supabase), Render

- **GitHub:** https://github.com/mingming-01/maple-growth-tracker
- **배포:** https://maple-growth-tracker.onrender.com

프로젝트 개요

메이플스토리 캐릭터의 성장 정보를 편하게 확인하고 싶어 시작한 개인 웹서비스다. NEXON Open API에서 캐릭터 데이터를 가져와 날짜별로 기록하고, 성장량과 랭킹 변화를 비교할 수 있도록 만들었다. 캐릭터 조회에서 끝내지 않고 성장 그래프, AI 리포트, 캐릭터 비교와 관리자 로그 기능까지 확장했다.

주요 수행 내용

- Flask에서 NEXON Open API를 호출하고 응답 데이터를 서비스 화면에 맞게 가공
- 캐릭터 기본 정보와 랭킹 등 조회 데이터 표시
- 날짜별 레벨·경험치·랭킹 기록 저장 및 조회
- 최근 기록뿐 아니라 30일·90일 기간의 성장 흐름을 살펴볼 수 있도록 과거 기록 수집 기능 확장
- 누락된 날짜를 구간으로 나누어 조회하고 이미 저장한 기록 재사용
- API에서 제공하지 않는 날짜와 API 오류 응답을 구분해 처리
- 경험치 증가량, 평균 성장량, 랭킹 변화 등을 계산하고 Chart.js로 시각화
- 최근 성장 흐름을 이용한 레벨업 예상치 계산
- Gemini API와 OpenRouter를 연결해 계산된 성장 지표를 자연어로 설명하는 AI 리포트 구현
- 두 캐릭터의 성장 지표와 기간별 기록을 비교하는 기능 구현
- 조회 결과를 PNG/PDF로 저장하고 관심 캐릭터를 관리하는 기능 구현
- 관리자 페이지에서 조회·AI 분석 로그를 확인하고 검색 조건에 따라 조회하도록 구성
- SQLite 및 PostgreSQL(Supabase) 연결을 구분하고 Render 배포 환경의 데이터 저장 상태 점검

문제 해결 및 검증

과거 성장 기록 수집

최근 기록만으로는 장기간의 성장 흐름을 비교하기 어려워 30일·90일 조회 기능을 확장했다. 누락된 날짜를 구간별로 나누어 요청하고 기존 저장 기록을 재사용하도록 구성했다. API가 데이터를 제공하지 않는 경우와 요청 자체가 실패한 경우를 구분해 잘못된 데이터를 만들지 않도록 했다.

배포 환경의 데이터 저장

로컬에서 동작하던 데이터 저장 방식이 배포 환경에서도 동일하게 유지되는 것은 아니었다. 환경변수와 데이터베이스 연결 설정을 구분하고 Render와 Supabase 연결 상태 및 실제 저장 결과를 확인했다.

AI 기능의 역할 분리

성장량과 통계 수치는 애플리케이션에서 계산하고, AI는 계산 결과를 자연어로 설명하도록 역할을 나눴다. 데이터가 부족한 상황에서 근거 없는 분석이 나오지 않도록 데이터의 유효성과 범위도 고려했다.

테스트 및 결과

Python unittest로 데이터 수집과 저장 로직을 확인하고, JavaScript 문법 검사 및 GitHub Actions를 활용했다. 정상 데이터, 데이터가 없는 날짜, API 오류 상황을 구분해 주요 동작을 검증했다. GitHub에 코드를 관리하고 Render에 배포해 실제 브라우저에서 사용할 수 있는 개인 웹서비스를 만들었다.

Packtory_DE — 제조·물류 업무를 다루는 MES형 시스템

- **유형:** ITWill 심화 과정 팀 프로젝트
- **기간:** 2025.03 ~ 2025.05
- **기술:** Java 21, Spring Boot, Gradle, JPA/Hibernate, MyBatis, QueryDSL, MySQL, H2, Thymeleaf, Spring Validation, Spring AOP
- **GitHub:** https://github.com/mingming-01/packtory_

프로젝트 개요

제조·MES 환경을 가정해 인사정보부터 제품, 원자재, 창고, 입출고, 품질검사와 근무관리까지 여러 업무를 연결한 생산관리 시스템이다. 기존 공통 구조를 이해한 뒤 기능을 추가하거나 수정하는 방식으로 개발했다.

주요 수행 내용

한 가지 기능에만 머무르지 않고 프로젝트 진행에 필요한 영역을 오가며 구현했다.

- **직원 관리:** 직원 등록·조회·수정·삭제 및 상세 화면 구현
- **부서 관리:** 대표 부서와 하위 부서를 코드로 연결하고 상위 부서 선택에 따라 하위 부서를 선택하도록 구성
- **부서 정보 표시:** DB에는 코드값을 저장하고 화면에는 한글 부서명을 표시하는 매핑 처리
- **직급 관리:** 직급 CRUD와 Controller, Service, Repository 및 Thymeleaf 화면 연결
- **제품 관리:** 기존 Item 관련 구조를 Product 중심으로 정리하고 DTO, VO, Service, Controller, Mapper와 화면 수정
- **원자재·창고·입출고:** 원자재, 창고, 위치정보, 창고 입출고 관련 기능 구현 및 화면 연결
- **품질 관리:** QCStandard와 Qc_LOG 등 품질 기준 및 검사 이력 관련 기능 작업
- **근무 관리:** WorkerMetrics, WorkerSchedule 등 근무 관련 기능과 등록된 스케줄의 목록 표시 기능 작업

개발 환경 및 데이터베이스 전환 경험

팀 서버를 이용해 로컬에서만 확인하지 않고 실제 서버 환경에서 기능을 시연하고 테스트했다. 프로젝트 후반에는 MySQL에서 Oracle로 전환하는 작업도 진행했지만, SQL과 설정 및 데이터 접근 부분의 수정 범위가 예상보다 커 최종 Oracle 연결까지 마무리하지는 못했다.

이 경험을 통해 DBMS를 바꾸는 작업은 접속 정보만 변경하는 것이 아니라 SQL, 드라이버, 설정과 데이터 처리 방식을 함께 검토해야 한다는 점을 배웠다.

배운 점

처음 접한 공통코드와 상·하위 코드 구조에 적응하면서 Controller, Service, DTO, Entity, Repository, Mapper와 Thymeleaf 화면이 연결되는 방식을 익혔다. JPA와 MyBatis를 함께 사용하는 구조도 경험했다. 기존 프로젝트의 코딩 방식과 데이터 흐름을 먼저 이해한 뒤 기능을 추가해야 팀원이 이어서 유지보수하기 쉽다는 점도 배웠다.

MrBean — 제조·생산관리 웹 프로젝트

- **유형:** ITWill 팀 프로젝트
- **기술:** Java 11, Spring MVC, MyBatis, MySQL, JSP, JSTL, JavaScript, HTML/CSS, Maven
- **GitHub:** https://github.com/shoqying/MrBean

프로젝트 개요

원자재, 완제품, BOM, 입고, 창고와 생산 관련 업무를 관리하는 웹 프로젝트다. Spring MVC와 MyBatis를 기반으로 JSP 화면에서 입력한 데이터가 Controller, Service, DAO와 Mapper를 거쳐 DB에 저장되고 조회되는 구조로 개발했다.

주요 수행 내용

팀원들과 필요한 기능을 나누어 작업했고, 저는 다음 영역을 중심으로 개발했다.

원자재 관리 - 원자재 코드, 이름, 원산지, 보관 방법 등의 정보 등록·조회·수정·삭제 - JSP 화면부터 Controller, Service, DAO, MyBatis Mapper까지 데이터 처리 연결

완제품 관리 - 완제품 코드, 이름, 설명 등의 등록·조회·수정·삭제 - 완제품 등록 시 BOM 목록을 선택해 BOM ID와 연결

BOM 관리 - BOM 등록, 목록·상세 조회, 수정, 삭제 - BOM 이름 중복 확인 - BOM ID 자동 생성 - 원자재 코드 연결 - BOM 비율을 0~100 범위로 검증 - BOM과 원자재 테이블을 JOIN해 연결된 원자재 이름 표시

원자재 입고 및 LOT 관리 - 입고 등록·조회·수정·삭제 - 날짜 기반 LOT 번호 생성 - 입고 데이터와 원자재 LOT 데이터 연결 - 원자재 코드, 수량, 단위, 유통기한 관리 - SQL에서 입고일 기준 6개월 뒤 유통기한 계산

창고 관련 작업 - 창고 목록 및 등록 화면 등 일부 JSP와 주변 기능 수정

문제 해결 및 배운 점

화면에 값이 표시되지 않거나 SQL 결과가 원하는 형태로 나오지 않을 때 JSP만 수정하는 데 그치지 않고 Controller, Service, DAO, Mapper와 SQL을 따라가며 원인을 찾았다. BOM과 완제품을 연결하고 원자재 입고와 LOT를 연결하면서 하나의 기능이 다른 기능의 데이터와 어떻게 이어지는지 함께 고려하는 습관을 익혔다.

Stock-Mate — QR·바코드 기반 재고 관리 시스템

- **유형:** ITWill 팀 프로젝트
- **기술:** Java 11, Spring Framework, MyBatis, MySQL, JSP, HTML5, CSS3, JavaScript, jQuery
- **GitHub:** https://github.com/in27sung/stock-mate

프로젝트 개요

상품과 재고, 창고의 입출고 과정을 관리하는 웹서비스다. 사용자, 사업장, 상품, 카테고리, 창고, 재고, 주문, 입고와 출고 데이터를 연결하고 QR·바코드로 상품을 식별하는 기능을 포함했다.

주요 수행 내용

특정 기능 하나만 맡기보다 필요한 영역을 함께 개발했고, 특히 Category와 Stock을 많이 작업했다.

Category - 카테고리 등록·목록 조회·수정·삭제 CRUD - 카테고리 관련 SQL 작성 및 수정 - Controller, Service, DAO, Mapper와 JSP 연결 - 카테고리 데이터 처리 오류 수정 - 등록·수정 화면 및 스타일 보완

Stock - 재고 목록 기능 수정 - StockController, StockService, DAO 및 구현체 수정 - Stock Mapper와 VO 수정 - 재고 관련 JSP 및 화면 데이터 연결

기타 - 사용자 정보 수정 과정의 오류 수정 - 로그인 관련 화면 일부 및 UserController 수정 - 공통 예외 처리와 설정 일부 수정 - Warehouse 관련 Mapper 수정

배운 점

카테고리는 상품에서 사용하고, 재고는 상품과 창고에 연결되며 주문·입출고도 같은 데이터를 참조한다. 한 기능을 수정할 때 해당 기능만 보는 것이 아니라 다른 기능에서 데이터를 어떻게 사용하는지 함께 고려해야 한다는 점을 배웠다. Controller부터 Service, DAO, Mapper, JSP까지 여러 계층을 직접 수정하며 Spring MVC와 MyBatis의 데이터 흐름을 익혔다.

T10 타이핑 연구 — 실험 설계 및 데이터 분석

- **유형:** ALEPH 연구 과제
- **주제:** 시각적 효과가 타이핑 성능에 미치는 영향
- **기술:** HTML, CSS, JavaScript, Python, CSV

프로젝트 개요

시각적 효과가 있는 경우와 없는 경우에 타이핑 성능 차이가 있는지 직접 실험하고 데이터를 분석했다.

주요 수행 내용

- JavaScript로 시각적 효과 ON/OFF 조건의 타이핑 실험 페이지 구성
- 총 10회 실험 진행 및 결과 CSV 저장
- Python으로 조건별 타이핑 속도와 정확도 분석
- 그래프로 결과를 시각화하고 처음 세운 가설과 실제 데이터 비교
- 실험 결과를 바탕으로 연구 내용과 결론 정리

분석 결과

현재 정리한 분석 결과는 다음과 같다.

지표	효과 OFF	효과 ON
평균 타이핑 속도	594.67	566.00
평균 정확도	100	99.47

이번 실험에서는 ON 조건의 평균 타이핑 속도와 정확도가 OFF 조건보다 낮게 나타났다. 다만 실험 횟수와 표본이 제한적이므로 모든 사용자에게 동일한 결과가 나타난다고 일반화하지 않고, 이번 실험에서 관찰한 결과로 해석했다.

배운 점

처음 세운 가설에 결과를 맞추기보다 실제 데이터를 기준으로 비교하고 결론을 내리는 경험을 했다. 실험을 재현할 수 있도록 데이터를 CSV로 남기고, Python 분석 코드와 그래프로 결과를 확인하는 과정도 익혔다. AI는 연구 주제와 코드 검토, 논문 정리에 보조적으로 활용했으며 실제 실험과 데이터 확인, 최종 판단은 직접 진행했다.

---

3. 기타 프로젝트 경험

My Profile — 개인 프로필 웹사이트

- GitHub: https://github.com/mingming-01/my-profile
- 배포: https://my-profile-ohhl.onrender.com/
- ALEPH T01에서 개인 프로필 페이지를 만들고, T08에서는 WebAuthn/Passkey를 활용한 Private Space 실습을 진행했다.
- Passkey 등록·로그인 challenge, 공개키 기반 서명 검증, Flask session을 이용한 접근 제어와 계정별 데이터 분리 테스트를 경험했다.
- 학습용 프로토타입이며, 실제 공개 서비스 수준의 보안 검증이 완료된 제품으로 소개하지 않는다.

PDS Diary — 계획·실행 기록 서비스

- 기술: Python, Flask, SQLAlchemy, SQLite, HTML/CSS/JavaScript, pytest
- Plan → Do → See → Improvement → Next Plan 흐름으로 계획과 실제 실행을 구분해 기록하는 개인 프로젝트
- Plan Version History, Task와 Execution 분리, 예상 시간과 실제 시간 비교, See 집계 및 근거 기록, JSON Export 구현
- 자동화 테스트 15개 통과 및 실제 데이터를 이용한 기능 검증
- 로그인 기능이 없는 공개 웹 애플리케이션이므로 민감한 개인 정보는 입력하지 않는 방식으로 사용

Arduino + Bluetooth + App Inventor 화재 알림 프로젝트

- Arduino와 센서, Bluetooth, MIT App Inventor를 연결해 센서 정보가 휴대폰에서 확인되는 화재 알림 시제품 구성
- 소프트웨어와 실제 장치를 함께 연결해 본 초기 프로젝트 경험

---

4. 역량별 경험 정리

백엔드 및 API

Flask와 Spring 기반 프로젝트에서 요청 처리, REST API, 인증, 데이터 조회·저장 기능을 구현했다. Maple Growth Tracker에서는 외부 API 데이터를 가공해 화면에 전달했고, SENTRA에서는 계정과 접근 정책을 처리하는 Flask API를 작업했다.

데이터베이스

MySQL, SQLite, H2를 프로젝트에서 사용했다. MyBatis Mapper와 SQL을 직접 다뤘고, JPA/Hibernate 및 SQLAlchemy 기반 데이터 처리도 경험했다. Maple Growth Tracker에서는 PostgreSQL(Supabase) 연결을 구성하고 배포 환경의 저장 상태를 확인했다.

인증 및 접근통제

SENTRA에서 JWT 기반 인증, 비활성 계정 차단, 직원 초대 가입, 역할·소속·자원별 접근 정책, 기간제 권한 요청·승인·회수 흐름을 구현·검증했다. My Profile의 T08에서는 WebAuthn/Passkey 인증과 session 기반 Private Space를 실습했다.

제조·업무 시스템

Packtory_DE, MrBean, Stock-Mate에서 직원·부서·직급, 제품, BOM, 원자재, 창고, 입출고, 품질 및 재고 업무를 다뤘다. 반도체공학 전공과 공정 실습 경험을 바탕으로 제조 업무 흐름과 IT 시스템을 연결하는 분야에 관심을 갖고 있다.

데이터 분석

T10에서 실험 페이지를 만들고 CSV 데이터를 수집해 Python으로 분석했다. 가설과 실제 결과를 비교하고, 표본 규모의 한계를 고려해 결론을 정리했다.

테스트 및 배포

Python unittest, pytest, JavaScript 문법 검사와 GitHub Actions 등을 활용했다. Render 및 GitHub Pages 배포 경험이 있으며, 배포 환경의 환경변수와 데이터베이스 연결 상태를 점검했다.

---

5. 지원 직무에 따른 경험 강조

백엔드 / 웹서비스 개발

SENTRA의 Flask API · JWT 인증·계정 및 권한 관리, Maple Growth Tracker의 외부 API 연동·데이터 저장·배포 경험을 중심으로 설명한다. ITWill 프로젝트에서는 Spring MVC, Spring Boot, MyBatis, JPA를 사용한 기능 구현 경험을 함께 제시한다.

MES / ERP / 제조 IT

반도체공학 전공과 공정 실습 경험을 바탕으로 Packtory_DE, MrBean, Stock-Mate에서 다룬 BOM · 원자재·입고·LOT · 창고·재고·생산 업무 기능을 연결해 설명한다. 교육 팀 프로젝트 경험과 실제 제조 현장 운영 경력은 구분한다.

IT 시스템 / 전산

SENTRA의 계정 상태 관리, 인증, 접근 정책, 감사 로그와 배포 환경 점검 경험을 중심으로 설명한다. Linux와 Bash는 교육·실습에서 다룬 범위를 구분해 제시한다.